Integer ALU Based Computing — FPGA Implementation Specification v1.0

Verilog Design for VFR Cores on Xilinx Zynq-7020

Registry: [CKS-MATH-145-2026]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [CKS-MATH-144-2026] → [CKS-MATH-145-2026]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [CKS-NEXT-1-2026]

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Overview

This specification defines the complete Verilog implementation of the VFR integer processor on Xilinx Zynq-7020 FPGA fabric. The design comprises 30 VFR cores, a batch dispatcher, shared read-only memory, and AXI bus interfaces connecting to the ARM processing system. The total design is approximately 1,650 lines of Verilog with 1,000 lines of testbench.

---

2. Design Parameters

Parameter Value Notes

———————— ————— —————

NUM_CORES 30 Configurable, limited by LUTs

CORE_BRAM_BYTES 9,216 2 BRAM tiles per core

ENTITIES_PER_CORE 38 At depth 48 (240 bytes/entity)

SHARED_BRAM_BYTES 24,576 3 BRAM tiles, read-only

CLK_FREQ_MHZ 150 Target clock frequency

INSTRUCTION_WIDTH 32 Fixed-width instructions

OPCODE_WIDTH 6 Supports 64 opcodes

V_REG_COUNT 16 i32 value registers

R_REG_COUNT 16 i8 remainder registers

V_REG_WIDTH 32 Matches i32

R_REG_WIDTH 8 Matches i8

AXI_DATA_WIDTH 64 AXI HP port width

AXI_ADDR_WIDTH 32 DDR3 address space

---

3. Module Hierarchy

fpga_top

├—— axi_registers ARM control/status interface (AXI GP slave)

├—— batch_dispatcher Work partitioning and DMA coordination

| └—— axi_dma_engine DDR3 read/write via AXI HP master

├—— shared_bram Read-only lookup tables (all cores read)

└—— vfr_core [0..29] 30 identical processing cores

├—— vfr_fetch Program counter and instruction read

├—— vfr_decode Opcode field extraction and control signals

├—— vfr_alu Arithmetic and logic unit

├—— vfr_registers V0-V15, R0-R15, batch control, flags

└—— core_bram Local 9KB block RAM (entity data)

---

4. Module Specifications

4.1 vfr_alu.v (~200 lines)

The ALU is purely combinational. No clock. No state. Opcode and operands go in, result comes out in the same cycle.

Ports:

verilog

```
module vfr_alu (
input wire [5:0] opcode,
input wire [31:0] operand_a, // First source (V register or immediate)
input wire [31:0] operand_b, // Second source (V register or immediate)
input wire [7:0] operand_ra, // First R register source
input wire [7:0] operand_rb, // Second R register source
input wire [17:0] immediate, // Immediate field from instruction
output reg [31:0] result_v, // V result
output reg [7:0] result_r, // R result
output reg flag_eq, // Equal flag
output reg flag_lt, // Less-than flag
output reg flag_gt, // Greater-than flag
output reg flag_ov, // Overflow flag
output reg write_v, // Result writes to V register
output reg write_r, // Result writes to R register
output reg write_flags // Result updates flags
);
```

Opcode Implementation:

verilog

```
always @(*) begin
// Defaults
result_v = 32'd0;
result_r = 8'd0;
flag_eq = 1'b0;
flag_lt = 1'b0;
```

```
flag_gt = 1'b0;
flag_ov = 1'b0;
write_v = 1'b0;
write_r = 1'b0;
write_flags = 1'b0;
case (opcode)
// ---- Arithmetic ----
6'h08: begin // ADD
    result_v = operand_a + operand_b;
    write_v = 1'b1;
end
6'h09: begin // SUB
    result_v = operand_a - operand_b;
    write_v = 1'b1;
end
6'h0A: begin // MUL (DSP48 inferred)
    result_v = operand_a * operand_b;
    write_v = 1'b1;
end
6'h0B: begin // ADDR
    result_r = operand_ra + operand_rb;
    write_r = 1'b1;
end
6'h0C: begin // SUBR
    result_r = operand_ra - operand_rb;
```

```
    write_r = 1'b1;
end
// ---- Bitwise ----
6'h10: begin // AND
    result_v = operand_a & operand_b;
    write_v = 1'b1;
end
6'h11: begin // OR
    result_v = operand_a | operand_b;
    write_v = 1'b1;
end
6'h12: begin // XOR
    result_v = operand_a ^ operand_b;
    write_v = 1'b1;
end
6'h13: begin // NOT
    result_v = ~operand_a;
    write_v = 1'b1;
end
// ---- Shift ----
6'h18: begin // SHR
    result_v = operand_a >> immediate[4:0];
    write_v = 1'b1;
```

```
end
6'h19: begin // SHL
    result_v = operand_a << immediate[4:0];
    write_v = 1'b1;
end
6'h1A: begin // SHR5
    result_v = operand_a >> 5;
    write_v = 1'b1;
end
6'h1B: begin // SHL5
    result_v = operand_a << 5;
    write_v = 1'b1;
end
// ---- VFR Operations ----
6'h20: begin // DECOMP: Vd = Va >> 5, Rd = Va & 0x1F
    result_v = operand_a >> 5;
    result_r = operand_a[4:0];
    write_v = 1'b1;
    write_r = 1'b1;
end
6'h21: begin // RECOMP: Vd = (Va << 5) | (Ra & 0x1F)
   result_v = (operand_a << 5) | {27'd0, operand_ra[4:0]};
    write_v = 1'b1;
```

```
end
6'h22: begin // RNORM: normalize remainder to -16..15
    result_r = operand_ra;
    if ($signed(operand_ra) > 31)
        result_r = operand_ra - 8'd32;
    if ($signed(operand_ra) < -32)
        result_r = operand_ra + 8'd32;
    write_r = 1'b1;
end
6'h23: begin // RACCUM: Rd = Ra + Rb, set OV
    result_r = operand_ra + operand_rb;
   flag_ov = ($signed(operand_ra + operand_rb) > 127) |
             ($signed(operand_ra + operand_rb) < -128);
    write_r = 1'b1;
    write_flags = 1'b1;
end
6'h24: begin // SCALE: Vd = Va << (imm * 5)
    result_v = operand_a << (immediate[4:0] * 5);
    write_v = 1'b1;
end
// ---- Compare ----
6'h28: begin // CMP
    flag_eq = (operand_a == operand_b);
```

```
   flag_lt = ($signed(operand_a) < $signed(operand_b));
   flag_gt = ($signed(operand_a) > $signed(operand_b));
    write_flags = 1'b1;
end
6'h29: begin // CMPR
    flag_eq = (operand_ra == operand_rb);
   flag_lt = ($signed(operand_ra) < $signed(operand_rb));
   flag_gt = ($signed(operand_ra) > $signed(operand_rb));
    write_flags = 1'b1;
end
default: begin
    result_v = 32'd0;
end
```

endcase

end

Note: Load/Store, Batch Control, Transfer, and Control Flow opcodes are not handled in the ALU. They are handled by the core's control logic in vfr_core.v. The ALU only computes arithmetic, bitwise, shift, VFR, and compare results.

4.2 vfr_decode.v (~150 lines)

Extracts instruction fields and generates control signals.

Ports:

verilog

module vfr_decode (

input wire [31:0] instruction,

output wire [5:0] opcode,

output wire [3:0] rd, // Destination register index

output wire [3:0] ra, // Source A register index

output wire [3:0] rb, // Source B register index

output wire [17:0] immediate,

// Control signals

output reg is_alu_op, // Goes to ALU

output reg is_load, // LDV, LDR, LDVR

output reg is_store, // STV, STR, STVR

output reg is_jump, // JMP, JEQ, JLT, JGT, JDONE

output reg is_batch, // BLOAD, BNEXT, BADDR

output reg is_transfer, // TSEND, TRECV, TWAIT, TDONE

output reg is_halt, // HALT

output reg is_vfr_pair, // LDVR/STVR: load/store both V and R

output reg uses_immediate // Instruction uses immediate field

);

Field Extraction:

verilog

// Fixed bit positions from ISA encoding spec

assign opcode = instruction[31:26];

assign rd = instruction[25:22];

assign ra = instruction[21:18];

assign rb = instruction[17:14];

assign immediate = instruction[17:0]; // Overlaps rb for immediate-type instructions

Control Signal Generation:

verilog

always @(*) begin

// Default all signals off

is_alu_op = 1'b0;

is_load = 1'b0;

is_store = 1'b0;

is_jump = 1'b0;

is_batch = 1'b0;

```
is_transfer = 1'b0;
is_halt = 1'b0;
is_vfr_pair = 1'b0;
uses_immediate = 1'b0;
casez (opcode)
6'h00: is_halt = 1'b1;                          // HALT
6'h01, 6'h02, 6'h03, 6'h04, 6'h05:          // JMP, JEQ, JLT, JGT, JDONE
    is_jump = 1'b1;
6'h08, 6'h09, 6'h0A, 6'h0B, 6'h0C:          // ADD, SUB, MUL, ADDR, SUBR
    is_alu_op = 1'b1;
6'h10, 6'h11, 6'h12, 6'h13:                // AND, OR, XOR, NOT
    is_alu_op = 1'b1;
6'h18, 6'h19: begin                           // SHR, SHL
    is_alu_op = 1'b1;
    uses_immediate = 1'b1;
end
6'h1A, 6'h1B:                                 // SHR5, SHL5
    is_alu_op = 1'b1;
6'h20, 6'h21, 6'h22, 6'h23:                // DECOMP, RECOMP, RNORM, RACCUM
    is_alu_op = 1'b1;
6'h24: begin                                       // SCALE
    is_alu_op = 1'b1;
    uses_immediate = 1'b1;
end
```

```
6'h28, 6'h29:                                   // CMP, CMPR
    is_alu_op = 1'b1;
6'h30, 6'h31: is_load = 1'b1;                    // LDV, LDR
6'h32, 6'h33: is_store = 1'b1;                   // STV, STR
6'h34: begin is_load = 1'b1; is_vfr_pair = 1'b1; end  // LDVR
6'h35: begin is_store = 1'b1; is_vfr_pair = 1'b1; end // STVR
6'h38, 6'h39, 6'h3A:                         // BLOAD, BNEXT, BADDR
    is_batch = 1'b1;
6'h3C, 6'h3D, 6'h3E, 6'h3F:                   // TSEND, TRECV, TWAIT, TDONE
    is_transfer = 1'b1;
endcase
end
```

4.3 vfr_registers.v (~100 lines)

Synchronous register file with two read ports and one write port.

Ports:

verilog

```
module vfr_registers (
input wire clk,
input wire reset,
// Read port A
input wire [3:0] read_addr_a,
output wire [31:0] read_data_va, // V register value
output wire [7:0] read_data_ra, // R register value
// Read port B
input wire [3:0] read_addr_b,
output wire [31:0] read_data_vb,
output wire [7:0] read_data_rb,
```

```
// Write port
input wire write_en_v,
input wire write_en_r,
input wire [3:0] write_addr,
input wire [31:0] write_data_v,
input wire [7:0] write_data_r,
// Batch control registers (directly accessible)
input wire write_en_batch,
input wire [15:0] write_bf, // Batch factor
input wire [31:0] write_bc, // Batch count
input wire [7:0] write_bd, // Batch depth
input wire [31:0] write_bi, // Batch index
output wire [15:0] bf,
output wire [31:0] bc,
output wire [7:0] bd,
output wire [31:0] bi,
// Memory address register
input wire write_en_ma,
input wire [31:0] write_ma,
output wire [31:0] ma,
// Flags
input wire write_en_flags,
input wire in_eq,
input wire in_lt,
input wire in_gt,
input wire in_ov,
input wire set_done,
output wire flag_eq,
output wire flag_lt,
output wire flag_gt,
output wire flag_ov,
```

output wire flag_done

);

Implementation:

verilog

reg [31:0] v_regs [0:15];

reg [7:0] r_regs [0:15];

reg [15:0] reg_bf;

reg [31:0] reg_bc;

reg [7:0] reg_bd;

reg [31:0] reg_bi;

reg [31:0] reg_ma;

reg reg_eq, reg_lt, reg_gt, reg_ov, reg_done;

// Read ports (combinational, no latency)

assign read_data_va = v_regs[read_addr_a];

assign read_data_ra = r_regs[read_addr_a];

assign read_data_vb = v_regs[read_addr_b];

assign read_data_rb = r_regs[read_addr_b];

assign bf = reg_bf;

assign bc = reg_bc;

assign bd = reg_bd;

assign bi = reg_bi;

assign ma = reg_ma;

assign flag_eq = reg_eq;

assign flag_lt = reg_lt;

assign flag_gt = reg_gt;

assign flag_ov = reg_ov;

assign flag_done = reg_done;

// Write port (synchronous)

always @(posedge clk) begin

if (reset) begin

```
integer i;
for (i = 0; i < 16; i = i + 1) begin
    v_regs[i] <= 32'd0;
    r_regs[i] <= 8'd0;
end
reg_bf <= 16'd0;
reg_bc <= 32'd0;
reg_bd <= 8'd0;
reg_bi <= 32'd0;
reg_ma <= 32'd0;
reg_eq <= 1'b0;
reg_lt <= 1'b0;
reg_gt <= 1'b0;
reg_ov <= 1'b0;
reg_done <= 1'b0;
```
end else begin
```
if (write_en_v) v_regs[write_addr] <= write_data_v;
if (write_en_r) r_regs[write_addr] <= write_data_r;
if (write_en_batch) begin
    reg_bf <= write_bf;
    reg_bc <= write_bc;
    reg_bd <= write_bd;
    reg_bi <= write_bi;
end
```

```
if (write_en_ma) reg_ma <= write_ma;
if (write_en_flags) begin
    reg_eq <= in_eq;
    reg_lt <= in_lt;
    reg_gt <= in_gt;
    reg_ov <= in_ov;
end
if (set_done) reg_done <= 1'b1;
```

end

end

4.4 vfr_fetch.v (~50 lines)

Program counter and instruction read from local BRAM.

Ports:

verilog

module vfr_fetch (

input wire clk,

input wire reset,

input wire enable, // Core is running

input wire stall, // Pipeline stall (memory wait)

// Jump control

input wire jump_en,

input wire [25:0] jump_addr,

// Halt

input wire halt,

// BRAM read interface

output wire [12:0] bram_addr, // Instruction address in BRAM

input wire [31:0] bram_data, // Instruction word

// Output

output reg [31:0] instruction,

output reg valid // Instruction is valid this cycle

);

Implementation:

verilog

reg [12:0] pc;

reg running;

assign bram_addr = pc;

always @(posedge clk) begin

if (reset) begin

```
pc <= 13'd0;

running <= 1'b0;

valid <= 1'b0;
```

end else if (halt) begin

```
running <= 1'b0;

valid <= 1'b0;
```

end else if (enable && !running) begin

```
running <= 1'b1;

pc <= 13'd0;

valid <= 1'b0;
```

end else if (running && !stall) begin

```
if (jump_en) begin

    pc <= jump_addr[12:0];

end else begin

    pc <= pc + 13'd1;

end
```

```
instruction <= bram_data;

valid <= 1'b1;
```

end

end

4.5 vfr_core.v (~300 lines)

Integrates all submodules and implements the 4-stage pipeline control logic. Handles load/store to local BRAM, batch control operations, jump evaluation, and halt.

Ports:

verilog

module vfr_core #(

parameter CORE_ID = 0,

parameter BRAM_DEPTH = 2304 // 9,216 bytes as 32-bit words

)(

input wire clk,

input wire reset,

// Control from dispatcher

input wire start, // Begin processing

output wire done, // Processing complete

output wire busy, // Currently running

// Local BRAM interface (for DMA loading by dispatcher)

input wire ext_bram_we, // External write enable

input wire [12:0] ext_bram_addr, // External address

input wire [31:0] ext_bram_wdata, // External write data

// Shared BRAM read interface

output wire [12:0] shared_bram_addr,

input wire [31:0] shared_bram_rdata

);

Pipeline Stages:

Stage 1 — Fetch:

Read instruction from BRAM at PC address.

Output: 32-bit instruction word.

Stage 2 — Decode:

Extract opcode, register indices, immediate.

Read source registers.

Generate control signals.

Output: opcode, operands, control signals.

Stage 3 — Execute:

ALU computes result (for ALU ops).

Address calculation (for load/store).

Jump condition evaluation (for branches).

Batch control logic (BLOAD reads 7 bytes, BNEXT increments BI, BADDR computes address).

Output: result value, memory address, jump target, flags.

Stage 4 — Writeback:

Write ALU result to destination register.

Write loaded value to register (for loads).

Update flags.

Update PC (for jumps).

Set done flag (for HALT).

Batch Control Logic (inside execute stage):

verilog

// BLOAD: read 7-byte header at MA

// Reads 3 memory locations:

// MA+0: BF (i16, sign-extended to 32 bits) — lower 16 bits

// MA+2: BC (u32)

// MA+6: BD (u8)

// Multi-cycle operation: stalls pipeline for 3 cycles

// BNEXT: BI += 1, set DONE if BI == BC

// Single cycle

// BADDR: Vd = MA + 7 + (BI * BD * 5)

// Multiply uses DSP48. Single cycle after multiply.

Jump Evaluation Logic:

verilog

// Evaluate jump conditions from flags

wire take_jump;

assign take_jump =

(opcode == 6'h01) | // JMP: always

(opcode == 6'h02 && flag_eq) | // JEQ: if equal

(opcode == 6'h03 && flag_lt) | // JLT: if less

(opcode == 6'h04 && flag_gt) | // JGT: if greater

(opcode == 6'h05 && flag_done); // JDONE: if batch done

Local BRAM (dual-port):

verilog

// Port A: core internal access (fetch + load/store)

// Port B: external access (dispatcher DMA loading)

// When dispatcher is loading, core is idle (start not asserted)

// No contention by design

reg [31:0] bram [0:BRAM_DEPTH-1];

// Port A: core read/write

always @(posedge clk) begin

if (core_bram_we)

bram[core_bram_addr] <= core_bram_wdata;

core_bram_rdata <= bram[core_bram_addr];

end

// Port B: external write (DMA)

always @(posedge clk) begin

if (ext_bram_we)

bram[ext_bram_addr] <= ext_bram_wdata;

end

4.6 batch_dispatcher.v (~400 lines)

The dispatcher is a state machine that reads batch parameters from the ARM's control registers, partitions work across cores, DMAs data from DDR3 to core BRAMs, signals cores to start, waits for completion, and DMAs results back.

Ports:

verilog

```
module batch_dispatcher #(
parameter NUM_CORES = 30,
parameter ENTITIES_PER_CORE = 38,
parameter ENTITY_BYTES = 240
)(
input wire clk,
input wire reset,
// Control registers (from axi_registers)
input wire [31:0] entity_addr, // DDR3 address of entity batch
input wire [31:0] entity_count, // Total entities to process
input wire [7:0] entity_depth, // Fields per entity
input wire [31:0] render_addr, // DDR3 address for results
input wire go, // Start signal from ARM
output reg busy,
output reg done,
output reg [31:0] cycle_count, // Profiling: total cycles
output reg [31:0] chunk_count, // Profiling: chunks processed
// Core control signals
output reg [NUM_CORES-1:0] core_start,
input wire [NUM_CORES-1:0] core_done,
// Core BRAM write interfaces (active during DMA load)
output reg [12:0] core_bram_addr [0:NUM_CORES-1],
output reg [31:0] core_bram_wdata [0:NUM_CORES-1],
output reg core_bram_we [0:NUM_CORES-1],
// AXI HP master interface (to DDR3)
```

output reg [31:0] m_axi_araddr, // Read address

output reg m_axi_arvalid,

input wire m_axi_arready,

input wire [63:0] m_axi_rdata, // Read data (64-bit)

input wire m_axi_rvalid,

output reg m_axi_rready,

output reg [31:0] m_axi_awaddr, // Write address

output reg m_axi_awvalid,

input wire m_axi_awready,

output reg [63:0] m_axi_wdata, // Write data

output reg m_axi_wvalid,

input wire m_axi_wready

);

State Machine:

verilog

localparam S_IDLE = 4'd0;

localparam S_CALC_CHUNKS = 4'd1;

localparam S_DMA_LOAD = 4'd2;

localparam S_DMA_LOAD_WAIT = 4'd3;

localparam S_DIST_TO_CORES = 4'd4;

localparam S_START_CORES = 4'd5;

localparam S_WAIT_CORES = 4'd6;

localparam S_DMA_STORE = 4'd7;

localparam S_DMA_STORE_WAIT= 4'd8;

localparam S_NEXT_CHUNK = 4'd9;

localparam S_DONE = 4'd10;

reg [3:0] state;

reg [31:0] total_chunks;

reg [31:0] current_chunk;

reg [31:0] chunk_entity_start;

reg [31:0] chunk_entity_count;

reg [31:0] cycle_counter;

State Transitions:

S_IDLE:

Wait for go signal.

On go: compute total_chunks = ceil(entity_count / (NUM_CORES * ENTITIES_PER_CORE))

→ S_CALC_CHUNKS

S_CALC_CHUNKS:

Compute this chunk's entity range.

chunk_entity_start = current_chunk * NUM_CORES * ENTITIES_PER_CORE

chunk_entity_count = min(remaining, NUM_CORES * ENTITIES_PER_CORE)

→ S_DMA_LOAD

S_DMA_LOAD:

Issue AXI read burst from entity_addr + chunk_entity_start * ENTITY_BYTES.

Read chunk_entity_count * ENTITY_BYTES from DDR3.

→ S_DMA_LOAD_WAIT

S_DMA_LOAD_WAIT:

Wait for AXI read to complete.

As data arrives, distribute to core BRAMs:

Entities 0..37 → core 0 BRAM

Entities 38..75 → core 1 BRAM

etc.

→ S_DIST_TO_CORES (or directly to S_START_CORES if DMA distributes inline)

S_START_CORES:

Assert core_start for all cores that have data.

(Last chunk may have fewer than NUM_CORES worth of entities.)

Start cycle counter.

→ S_WAIT_CORES

S_WAIT_CORES:

Poll core_done bits.

Increment cycle counter each clock.

When all active cores report done:

→ S_DMA_STORE

S_DMA_STORE:

Read render-relevant fields from each core's BRAM.

Issue AXI write burst to render_addr + chunk_entity_start * RENDER_BYTES.

Only 5 fields × 5 bytes = 25 bytes per entity (position, sprite, anim state).

→ S_DMA_STORE_WAIT

S_DMA_STORE_WAIT:

Wait for AXI write to complete.

→ S_NEXT_CHUNK

S_NEXT_CHUNK:

current_chunk += 1

chunk_count += 1

if current_chunk < total_chunks:

→ `S_CALC_CHUNKS`

else:

→ `S_DONE`

S_DONE:

Set done flag.

Accumulate cycle_counter into cycle_count.

→ S_IDLE (wait for next go)

4.7 axi_registers.v (~200 lines)

Memory-mapped register bank accessible by the ARM via AXI GP slave port.

Register Map:

verilog

// Address Name Access Width Description

// 0x00 CONTROL W 1 bit Write 1 to start, auto-clears

// 0x04 STATUS R 3 bits [0]=busy, [1]=done, [2]=error

// 0x08 ENTITY_ADDR R/W 32 bits DDR3 address of entity batch

// 0x0C ENTITY_COUNT R/W 32 bits Number of entities

// 0x10 ENTITY_DEPTH R/W 8 bits Fields per entity

// 0x14 RENDER_ADDR R/W 32 bits DDR3 address for render output

// 0x18 RENDER_FIELDS R/W 8 bits Number of fields to write back

// 0x1C SM_ADDR R/W 32 bits State machine batch address

// 0x20 BEHAVIOR_ADDR R/W 32 bits Behavior set batch address

// 0x24 FACT_SCHEMA_ADDR R/W 32 bits Fact generation rules address

// 0x28 ENVELOPE_ADDR R/W 32 bits Envelope batch address

// 0x2C SHARED_LOAD_ADDR R/W 32 bits Source address for shared BRAM load

// 0x30 SHARED_LOAD_SIZE R/W 32 bits Bytes to load into shared BRAM

// 0x34 SHARED_LOAD_GO W 1 bit Trigger shared BRAM load

// 0x38 CORE_COUNT R 8 bits Number of VFR cores (hardwired)

// 0x3C CORE_STATUS R 32 bits Per-core done/busy bits

// 0x40 CYCLE_COUNT R 32 bits Cycles for last batch

// 0x44 CHUNK_COUNT R 32 bits Chunks processed in last batch

AXI Slave Implementation:

Standard AXI4-Lite slave pattern. On AXI write: decode address, store value into register. On AXI read: decode address, return register value. Vivado provides templates for this — the pattern is identical for every AXI register bank. The only custom part is the register map.

4.8 shared_bram.v (~50 lines)

Dual-port block RAM. Port A is written by the dispatcher (loading data from DDR3 via AXI). Port B is read by all cores through a multiplexed read bus.

verilog

module shared_bram #(

parameter DEPTH = 6144 // 24 KB as 32-bit words

)(

input wire clk,

// Port A: write (from dispatcher/DMA)

input wire we_a,

input wire [12:0] addr_a,

input wire [31:0] din_a,

// Port B: read (from cores, active core selected externally)

input wire [12:0] addr_b,

output reg [31:0] dout_b

);

reg [31:0] mem [0:DEPTH-1];

always @(posedge clk) begin

if (we_a)

mem[addr_a] <= din_a;

end

always @(posedge clk) begin

dout_b <= mem[addr_b];

end

endmodule

Core read access is arbitrated round-robin or the dispatcher stalls reads during DMA writes. In practice, shared BRAM is loaded once at scene start and is read-only during frame processing, so there is no contention.

Contents:

Address Range Content Size

0x0000 - 0x00FF Perlin permutation table 256 × 4 = 1,024 bytes

0x0100 - 0x01FF Perlin fade table 256 × 4 = 1,024 bytes

0x0200 - 0x09FF UtilityAI curve tables 8 curves × 256 × 4 = 8,192 bytes

0x0A00 - 0x0FFF State machine data ~6,144 bytes

0x1000 - 0x17FF Behavior set data ~8,192 bytes

Total: ~24,576 bytes (3 BRAM tiles)

4.9 fpga_top.v (~200 lines)

Top-level module that instantiates all components and wires them together.

verilog

module fpga_top #(

parameter NUM_CORES = 30

)(

// AXI GP Slave interface (from ARM, control registers)

```
input wire s_axi_aclk,
input wire s_axi_aresetn,
input wire [31:0] s_axi_awaddr,
input wire s_axi_awvalid,
output wire s_axi_awready,
input wire [31:0] s_axi_wdata,
input wire s_axi_wvalid,
output wire s_axi_wready,
output wire [1:0] s_axi_bresp,
output wire s_axi_bvalid,
input wire s_axi_bready,
input wire [31:0] s_axi_araddr,
input wire s_axi_arvalid,
output wire s_axi_arready,
output wire [31:0] s_axi_rdata,
output wire [1:0] s_axi_rresp,
output wire s_axi_rvalid,
input wire s_axi_rready,
// AXI HP Master interface (to DDR3)
output wire [31:0] m_axi_araddr,
output wire m_axi_arvalid,
input wire m_axi_arready,
input wire [63:0] m_axi_rdata,
input wire m_axi_rvalid,
output wire m_axi_rready,
output wire [31:0] m_axi_awaddr,
output wire m_axi_awvalid,
input wire m_axi_awready,
output wire [63:0] m_axi_wdata,
output wire m_axi_wvalid,
input wire m_axi_wready
```

```
);
// Internal wires: control registers to dispatcher
wire [31:0] entity_addr;
wire [31:0] entity_count;
wire [7:0] entity_depth;
wire [31:0] render_addr;
wire go;
wire busy;
wire done;
wire [31:0] cycle_count;
wire [31:0] chunk_count;
// Per-core signals
wire [NUM_CORES-1:0] core_start;
wire [NUM_CORES-1:0] core_done;
wire [NUM_CORES-1:0] core_busy;
// Instantiate register bank
axi_registers regs (
.clk(s_axi_aclk),
.reset(~s_axi_aresetn),
// AXI slave ports...
// Register outputs to dispatcher...
.entity_addr(entity_addr),
.entity_count(entity_count),
.entity_depth(entity_depth),
.render_addr(render_addr),
.go(go),
.busy(busy),
```

```
.done(done),
.core_count(NUM_CORES),
.core_status({core_done, core_busy}),
.cycle_count(cycle_count),
.chunk_count(chunk_count)
);
// Instantiate shared BRAM
wire [12:0] shared_addr_mux;
wire [31:0] shared_rdata;
shared_bram #(.DEPTH(6144)) shared_data (
.clk(s_axi_aclk),
.we_a(shared_we),
.addr_a(shared_write_addr),
.din_a(shared_write_data),
.addr_b(shared_addr_mux),
.dout_b(shared_rdata)
);
// Instantiate VFR cores
genvar i;
generate
for (i = 0; i < NUM_CORES; i = i + 1) begin : gen_cores
    vfr_core #(
        .CORE_ID(i),
        .BRAM_DEPTH(2304)
    ) core (
        .clk(s_axi_aclk),
```

```
        .reset(~s_axi_aresetn),
        .start(core_start[i]),
        .done(core_done[i]),
        .busy(core_busy[i]),
        .ext_bram_we(dispatcher_bram_we[i]),
        .ext_bram_addr(dispatcher_bram_addr[i]),
        .ext_bram_wdata(dispatcher_bram_wdata[i]),
        .shared_bram_addr(core_shared_addr[i]),
        .shared_bram_rdata(shared_rdata)
    );
end
endgenerate
// Instantiate batch dispatcher
batch_dispatcher #(
.NUM_CORES(NUM_CORES),
.ENTITIES_PER_CORE(38)
) dispatcher (
.clk(s_axi_aclk),
.reset(~s_axi_aresetn),
.entity_addr(entity_addr),
.entity_count(entity_count),
.entity_depth(entity_depth),
.render_addr(render_addr),
.go(go),
```

```
.busy(busy),

.done(done),

.cycle_count(cycle_count),

.chunk_count(chunk_count),

.core_start(core_start),

.core_done(core_done),

// AXI HP master ports...

// Core BRAM write interfaces...
);
endmodule
```

---

5. Vivado IP Integrator Block Design

The Vivado project uses a block design that connects the hard Zynq PS to the custom FPGA logic. This is configured graphically in Vivado's IP Integrator.

Blocks in the design:

ZYNQ7 Processing System (PS)

├── DDR3 memory controller (auto-configured for Zybo board)

├── AXI GP0 Master → AXI Interconnect → axi_registers (our register bank)

├── AXI HP0 Slave ← batch_dispatcher (our DMA engine reads/writes DDR3)

├── UART (serial debug, directly from PS)

├── USB (keyboard/mouse, directly from PS)

├── GEM Ethernet (network, directly from PS)

├── SDIO (SD card, directly from PS)

└── HDMI (directly from PS via EMIO or fabric)

fpga_top (our custom logic)

├── Connected to AXI GP0 as slave (control registers)

└── Connected to AXI HP0 as master (DDR3 DMA)

The PS peripherals (UART, USB, GEM, SDIO) connect directly to the ARM and do not involve the FPGA fabric at all. The Zig kernel accesses them through memory-mapped ARM peripheral registers, exactly as documented in the Silo OS spec.

The HDMI output can be driven from the PS side through EMIO pins or through a simple framebuffer-to-HDMI IP core in the fabric. For the Zybo Z7, Digilent provides an HDMI transmitter IP that takes a framebuffer address in DDR3 and outputs to the HDMI connector. This runs independently of the VFR cores.

---

6. Simulation Plan

6.1 tb_alu.v — ALU Unit Test

Tests every opcode with known input/output pairs:

Test: ADD 0x00000005, 0x00000003 → expect 0x00000008

Test: SUB 0x00000005, 0x00000003 → expect 0x00000002

Test: MUL 0x00000005, 0x00000003 → expect 0x0000000F

Test: AND 0x0000FFFF, 0x00FF00FF → expect 0x0000FF00… wait, 0x00FF → 0x00FF

Test: SHR5 0x00000020 → expect 0x00000001

Test: SHL5 0x00000001 → expect 0x00000020

Test: DECOMP 0x00000027 → V=0x00000001, R=0x07

Test: RECOMP V=0x00000001, R=0x07 → 0x00000027

Test: CMP 5, 3 → EQ=0, LT=0, GT=1

Test: CMP 3, 5 → EQ=0, LT=1, GT=0

Test: CMP 5, 5 → EQ=1, LT=0, GT=0

… all 35 opcodes with edge cases (zero, negative, overflow)

Run count: ~200 test vectors. Simulation time: milliseconds.

6.2 tb_core.v — Single Core Integration Test

Loads a small program into simulated BRAM and runs the core:

Program: Add two entities' health values

BRAM contents:

Address 0x000: BLOAD instruction (load batch header)

Address 0x004: BADDR V0 (compute entity address)

Address 0x008: LDVR V1, R1, [V0] (load first entity health)

Address 0x00C: LDVR V2, R2, [V0+5] (load second value)

Address 0x010: ADD V3, V1, V2

Address 0x014: STVR [V0+10], V3, R3 (store result)

Address 0x018: BNEXT

Address 0x01C: JDONE 0x024

Address 0x020: JMP 0x004

Address 0x024: HALT

Batch header at data region:

F=1, count=4, depth=3

Entity data:

Entity 0: [10, 0] [20, 0] [0, 0]

Entity 1: [30, 0] [40, 0] [0, 0]

Entity 2: [50, 0] [60, 0] [0, 0]

Entity 3: [70, 0] [80, 0] [0, 0]

Expected output:

Entity 0: [10, 0] [20, 0] [30, 0]

Entity 1: [30, 0] [40, 0] [70, 0]

Entity 2: [50, 0] [60, 0] [110, 0]

Entity 3: [70, 0] [80, 0] [150, 0]

Verify: all batch control registers update correctly, DONE flag sets after 4 iterations, HALT stops the core.

6.3 tb_dispatcher.v — Batch Dispatcher Test

Simulates the dispatcher with a mock AXI memory and 4 cores:

Setup:

Mock DDR3 at address 0x02000000 containing 100 entities

entity_count = 100

NUM_CORES = 4

ENTITIES_PER_CORE = 38

Expected behavior:

Chunk 0: entities 0-99 (100 total)

Core 0: entities 0-24 (25)

Core 1: entities 25-49 (25)

Core 2: entities 50-74 (25)

Core 3: entities 75-99 (25)

DMA load: 100 * 240 = 24,000 bytes read from mock DDR3

Core start: all 4 cores signaled

Core done: all 4 cores report complete

DMA store: 100 * 25 = 2,500 bytes written to render_addr

Status: done = 1

6.4 tb_system.v — Full System Test

Simulates the complete fpga_top with mock AXI bus, DDR3, and ARM register writes:

Sequence:

1. Write entity_addr = 0x02000000
2. Write entity_count = 1000
3. Write entity_depth = 48
4. Write render_addr = 0x03000000
5. Write CONTROL = 1

Monitor: busy goes high, chunk processing proceeds, done goes high

Read: cycle_count, chunk_count

Verify: render output at 0x03000000 matches expected values

---

7. Resource Utilization Estimate

Component LUTs FFs BRAM DSP48

———————— ———— ———— ———— ————

vfr_core × 30 42,000 42,000 60 30

(1,400 per core)

batch_dispatcher 2,000 1,500 0 0

axi_registers 500 400 0 0

shared_bram 100 50 3 0

axi_dma_engine 1,500 1,000 0 0

interconnect 500 300 0 0

vivado infrastructure 500 300 2 0

Total 47,100 45,550 65 30

Available (Zynq-7020) 53,200 106,400 140 220

Utilization 88.5% 42.8% 46.4% 13.6%

LUTs are the limiting resource at 88.5%. If timing closure requires more routing space, reduce to 28 cores. FFs, BRAM, and DSP48 have ample headroom.

---

8. Timing Targets

Target clock: 150 MHz (6.67 ns period)

Critical paths (estimated):

ALU multiply: ~4 ns (DSP48 handles this)

BRAM read: ~2 ns (single-cycle synchronous read)

Register file read: ~1 ns (small LUT-based RAM)

Instruction decode: ~1.5 ns (combinational case statement)

Worst case pipeline stage: execute (ALU + mux) at ~5 ns

Margin: 6.67 - 5.0 = 1.67 ns slack

If timing fails at 150 MHz:

Option 1: Reduce to 125 MHz (safe, ~10% performance loss)

Option 2: Add pipeline register in ALU output path (1 cycle latency, keeps 150 MHz)

Option 3: Reduce core count to ease routing congestion

---

9. ARM Software Interface (Zig)

The Zig kernel interacts with the FPGA through memory-mapped register writes:

zig

const FPGA_BASE: u32 = 0x4000_0000;

const Reg = struct {

```
const CONTROL: *volatile u32 = [?](FPGA_BASE + 0x00);
const STATUS: *volatile u32 = [?](FPGA_BASE + 0x04);
const ENTITY_ADDR: *volatile u32 = [?](FPGA_BASE + 0x08);
const ENTITY_COUNT: *volatile u32 = [?](FPGA_BASE + 0x0C);
const ENTITY_DEPTH: *volatile u32 = [?](FPGA_BASE + 0x10);
const RENDER_ADDR: *volatile u32 = [?](FPGA_BASE + 0x14);
const RENDER_FIELDS: *volatile u32 = [?](FPGA_BASE + 0x18);
const SM_ADDR: *volatile u32 = [?](FPGA_BASE + 0x1C);
const BEHAVIOR_ADDR: *volatile u32 = [?](FPGA_BASE + 0x20);
const FACT_SCHEMA_ADDR:*volatile u32 = [?](FPGA_BASE + 0x24);
const ENVELOPE_ADDR: *volatile u32 = [?](FPGA_BASE + 0x28);
const CORE_COUNT: *volatile u32 = [?](FPGA_BASE + 0x38);
const CORE_STATUS: *volatile u32 = [?](FPGA_BASE + 0x3C);
const CYCLE_COUNT: *volatile u32 = [?](FPGA_BASE + 0x40);
const CHUNK_COUNT: *volatile u32 = [?](FPGA_BASE + 0x44);
};
fn dispatchEntityBatch(
entity_ddr3_addr: u32,
entity_count: u32,
render_ddr3_addr: u32,
) void {
Reg.ENTITY_ADDR.* = entity_ddr3_addr;
Reg.ENTITY_COUNT.* = entity_count;
Reg.ENTITY_DEPTH.* = 48;
Reg.RENDER_ADDR.* = render_ddr3_addr;
Reg.CONTROL.* = 1;
// Poll for completion
while (Reg.STATUS.* & 0x02 == 0) {
// ARM can do other work here (input, network)
```

```
}
// Read profiling
const cycles = Reg.CYCLE_COUNT.*;
const chunks = Reg.CHUNK_COUNT.*;
}
fn loadSharedData(source_addr: u32, size: u32) void {
const shared_load_addr: *volatile u32 = [?](FPGA_BASE + 0x2C);
const shared_load_size: *volatile u32 = [?](FPGA_BASE + 0x30);
const shared_load_go: *volatile u32 = [?](FPGA_BASE + 0x34);
shared_load_addr.* = source_addr;
shared_load_size.* = size;
shared_load_go.* = 1;
// Wait for load complete
while (Reg.STATUS.* & 0x04 != 0) {}
}
```

---

10. SD Card Boot Image

The SD card contains a single BOOT.bin created by Vivado's bootgen tool:

BOOT.bin contains (in order):

1. FSBL (First Stage Boot Loader)
   - Generated automatically by Vivado from the hardware design
   - Initializes DDR3, clocks, MIO pins
   - Programs FPGA with bitstream
   - Jumps to kernel
2. FPGA bitstream (fpga_top.bit)
   - Our VFR core design
   - Loaded into PL by FSBL
3. Silo OS kernel (silo_kernel.elf)
   - Compiled Zig bare-metal binary
   - ARM Cortex-A9 target

- Entry point at 0x00100000

Bootgen BIF file:

the_ROM_image:

{

[bootloader] fsbl.elf

[destination_device=pl] fpga_top.bit

[destination_cpu=a9-0] silo_kernel.elf

}

Command: `bootgen -image boot.bif -o BOOT.bin -w`

The resulting BOOT.bin is copied to the root of the SD card's FAT32 partition. On power-up, the Zynq boot ROM reads BOOT.bin automatically.

---

11. Development Milestones

Week Milestone Deliverable Test Method

——— ——————— ——————— ———————

1-2 ALU verified vfr_alu.v + tb_alu.v Simulation

3-4 Single core verified vfr_core.v + tb_core.v Simulation

5-6 Vivado project, 1 core on HW fpga_top.bit ARM C test program

7-8 Batch dispatcher + 4 cores batch_dispatcher.v ARM dispatches test batch

9-10 Scale to 30 cores Full fpga_top Performance profiling

11-12 Port ARM to Zig bare-metal silo_kernel.elf Zig dispatches batch, UART output

13-14 Shared BRAM + Silo entity pipeline Lookup tables loaded Entity processing verified

15-16 Beam-casting + rendering SVDAG in shared BRAM Pixels on HDMI display

17-20 Full integration + optimization Complete system 60 FPS game demo

---

12. Files Checklist

File Lines Purpose Status

——— ——— ——— ———

rtl/vfr_alu.v 200 Arithmetic and logic unit To build

rtl/vfr_decode.v 150 Instruction decoder To build

rtl/vfr_registers.v 100 Register file To build

rtl/vfr_fetch.v 50 Program counter + instruction read To build

rtl/vfr_core.v 300 Core integration + pipeline control To build

rtl/batch_dispatcher.v 400 Work partitioning + DMA To build

rtl/axi_registers.v 200 ARM control/status register bank To build

rtl/shared_bram.v 50 Read-only shared lookup tables To build

rtl/fpga_top.v 200 Top-level instantiation To build

sim/tb_alu.v 300 ALU test vectors To build

sim/tb_core.v 300 Single core program test To build

sim/tb_dispatcher.v 200 Batch dispatcher test To build

sim/tb_system.v 200 Full system integration test To build

constraints/zybo.xdc — Pin constraints (download from Digilent)

scripts/build.tcl — Vivado batch build script To build

scripts/program.tcl — Vivado programming script To build

Total Verilog: 1,650 lines

Total Testbench: 1,000 lines

Total: 2,650 lines

---

13. Summary

Property	Specification
Target device	Xilinx Zynq-7020 (Zybo Z7-20 board)
Design language	Verilog
Total Verilog	~1,650 lines
Total testbench	~1,000 lines
VFR cores	30 at 150 MHz
Core pipeline	4-stage in-order (fetch, decode, execute, writeback)
LUT utilization	~88% (47,100 / 53,200)
BRAM utilization	~46% (65 / 140)
DSP48 utilization	~14% (30 / 220)
ARM interface	AXI GP slave (registers) + AXI HP master (DMA)
ARM software	Zig bare-metal kernel, memory-mapped register access

Property	Specification
Boot method	SD card BOOT.bin (FSBL + bitstream + kernel)
Debug	Vivado simulator, UART serial, ILA optional
Development time	~20 weeks from start to integrated demo

Integer ALU Based Computing — FPGA Implementation Specification v1.0

Companion document to ISA v1.0, Compiler v1.0, Silo-VFR Mapping v1.0, Motherboard v2.0, and Rendering Pipeline v2.0

[**CKS-MATH-145-2026**] Integer ALU Based Computing — FPGA Implementation Specification v1.0. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-145-2026

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C
MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS
MATH-104-2026

[**CKS-MATH-144-2026**] Integer ALU Based Computing — Rendering Pipeline Specification v2.0. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-144-2026

[**CKS-NEXT-1-2026**] Post-CKS. CKS is invalidated and falsified. Github: https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/paper
LEX-12-2026